Video-Based Fruit Counting with Tracking to Eliminate Double-Counting

Derrick Samuel R. Prannavakhanth Priyanka Kaliraj
Vellore Institute of Technology, Chennai

Abstract

Counting fruit from orchard video requires deciding, for every detection, whether it belongs to a fruit that has already been counted. Detector-only counting double-counts, and short-term trackers fragment identities when fruit are occluded. We present OrchardVision, an end-to-end system for mango fruit tracking in video. It combines YOLO detection, interchangeable trackers (a centre-distance baseline, ByteTrack, and TAPNext), optional camera-motion compensation, and an identity memory that stores a prototype embedding per fruit so that a track that reappears after occlusion can be re-associated with an existing identity. Every intermediate result is persisted (SQLite database, local-disk storage) and exposed through a REST API and an experiment harness. We evaluate the trackers on three short orchard videos against a *provisional* ground truth that was generated automatically by the same detector and has not been confirmed by a human annotator; the identity metrics therefore measure consistency with that reference, not absolute counting accuracy. With ByteTrack, the predicted track count exceeded the provisional ground truth on all three videos (absolute errors of 1, 11, and 3 on ground-truth counts of 9, 7, and 14). Camera-motion compensation changed mean IDF1 and HOTA by 0.3 and 0.6 points while raising mean identity switches from 5.7 to 17.3. The TAPNext results and the ReID-threshold sweep are unresolved and are not interpreted; the identity memory is not part of the evaluated loop, and cross-session identity matching is left as future work.

Keywords: fruit counting; multi-object tracking; re-identification; precision agriculture; mango orchard.

1 Introduction

Fruit count is central to yield forecasting, harvest logistics, and crop-load management, yet it is still commonly estimated by hand-counting a small sample of trees and extrapolating. Because fruit load varies strongly between trees and rows, such sampling is slow and statistically weak. Video recorded by a handheld, vehicle-mounted, or aerial camera samples an orchard far more densely, but converting it into a count is not simply a matter of running a detector on every frame: a fruit is visible in dozens or hundreds of consecutive frames and is reported each time. The core problem is one of *identity*: is a detection in the current frame a fruit that has already been counted, or a new one?

The standard remedy is tracking-by-detection, in which a tracker links detections into trajectories and the count is the number of trajectories. This works when fruit stay visible and the camera moves smoothly. In orchards, foliage hides fruit for tens of frames, which breaks trajectories and inflates the count when the fruit reappears; handheld or vehicle-mounted cameras move constantly, which disturbs motion models; and fruit on the same tree are visually similar, which weakens appearance cues. Most tracking research was developed for pedestrians and vehicles, not for dense, small, near-identical objects.

This paper describes OrchardVision, a system that adds an *identity memory* to a tracking pipeline: each fruit is represented by a prototype embedding with quality and observation statistics, and a track that appears after another has been lost can be linked back to the lost identity if appearance and motion both allow it. Among the works reviewed in Section 2, we found none that persists tracks, identity prototypes, re-identification events, per-frame timing, and camera-motion estimates in a queryable form together with a harness for repeatable tracker comparison on the same video; OrchardVision is built to provide this. The contributions are:

1. a pipeline design that couples detection, a pluggable tracker, camera-motion compensation, and a prototype-based identity memory, with every intermediate result persisted so that a counting error can be traced to the stage that produced it (Section 3);
2. an experiment harness and offline evaluation scripts that compare tracker back-ends with IDF1, HOTA, identity switches, and count error (Sections 4 and 5);
3. an evaluation on three orchard videos covering detector selection, a tracker comparison, a camera-motion ablation, a detection-threshold sweep, counting error, and runtime; and
4. an explicit account of what that evaluation does *not* establish (Section 6).

Scope of the claims. The system is implemented and runs end to end. The evaluation uses a machine-generated ground truth and three short videos, so it supports statements about tracker consistency on these videos, not about absolute counting accuracy. The identity memory is not part of the evaluated loop, the ReID-threshold sweep and TAPNext results are unresolved, and matching across recording sessions or days has not been evaluated.

2 Related Work

Fruit detection. Deep detectors have replaced hand-crafted colour and shape features for fruit detection. Bargoti and Underwood [2] applied a Faster R-CNN detector to orchard imagery of mangoes, apples, and almonds, and Koirala et al. [11] survey deep-learning methods for fruit detection and yield estimation. Single-stage detectors of the YOLO family [17] are widely used for their speed; we use the maintained Ultralytics implementation [21]. Per-frame detection alone carries no temporal identity.

Tracking-by-detection. SORT [5] associates detections across frames using a Kalman filter [10] for motion prediction and the Hungarian algorithm [12] for assignment; DeepSORT [20] adds a learned appearance embedding to the association cost. ByteTrack [22] also associates low-confidence detections, which often correspond to partly occluded objects, to recover tracks that would otherwise be lost; this is relevant to fruit, where partial occlusion lowers detector confidence. BoT-SORT [1] combines motion and appearance cues and adds camera-motion compensation, in which global image motion is estimated and used to correct motion predictions. These methods were designed and evaluated mainly for pedestrians.

Re-identification and point tracking. Re-identification maps an image crop to an embedding in which crops of the same identity are close; Hermans et al. [8] show that a carefully mined triplet loss is a strong baseline for person re-identification. An identity can be summarised by a prototype

that aggregates its observations, and new detections can be compared with prototypes. Point-tracking methods follow a query point through occlusion; TAP-Vid [6] is a benchmark for this task and TAPNext [23] formulates it as next-token prediction. Tracking a point on a fruit surface could, in principle, be more stable under occlusion than tracking a bounding box; we treat TAPNext as one tracker option and do not interpret its results here (Section 5).

Fruit counting from video. Liu et al. [13] count fruit with a monocular camera by tracking fruit and associating them with a three-dimensional map to avoid double-counting. This requires structure from motion, which can be brittle in foliage-rich scenes. OrchardVision takes a complementary, appearance- and motion-based route that needs no three-dimensional reconstruction.

Metrics. The CLEAR MOT metrics [4] count identity switches (IDSW), false positives, and misses. IDF1 [18] measures how long identities are correctly maintained:

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}, \tag{1}$$

where IDTP, IDFP, and IDFN come from a global one-to-one matching between ground-truth and predicted trajectories. HOTA [14] balances detection and association quality,

$$\mathrm{HOTA}_\beta = \sqrt{\mathrm{DetA}_\beta \cdot \mathrm{AssA}_\beta}, \tag{2}$$

and is usually averaged over localisation thresholds β. For counting, every identity switch changes the count, so IDF1 and IDSW are especially informative.

3 System and Method

3.1 Overview

A user uploads an orchard video; the backend processes it in a single streaming pass and persists the results. Figure 1 shows the stages: frame extraction and detection, tracking (with optional camera-motion compensation), appearance embedding, identity consolidation, and analytics. The backend is a FastAPI [15] service with a SQLAlchemy [3] 2.0 data layer, and the web client is a React and TypeScript application. Long jobs run as a background task or through a Celery queue. The database is SQLite.

Uploaded videos and cached JPEG frames are stored on the *local file system* through a storage service that confines resolved paths to the upload directory. Object storage is not implemented. **[Author check:** the source abstract claims S3-compatible object storage; the source deployment section states local disk only and that object storage is not implemented. This paper follows the deployment section.**]**

Detections, tracks, identity prototypes, per-sighting observations, re-identification events, per-frame timings, and camera-motion estimates are stored as separate records, so that a counting error can be traced to the stage that produced it. Each processing job stores the configuration it ran with, and the experiment harness stores each run's tracker, scenario, and metrics and can export them as JSON or CSV.

3.2 Detection

Fruit are detected by a YOLO model [17, 21]. Detections below a confidence threshold τ_{conf} (default 0.35) are discarded; detections with confidence below 0.45 are additionally flagged as occluded. A

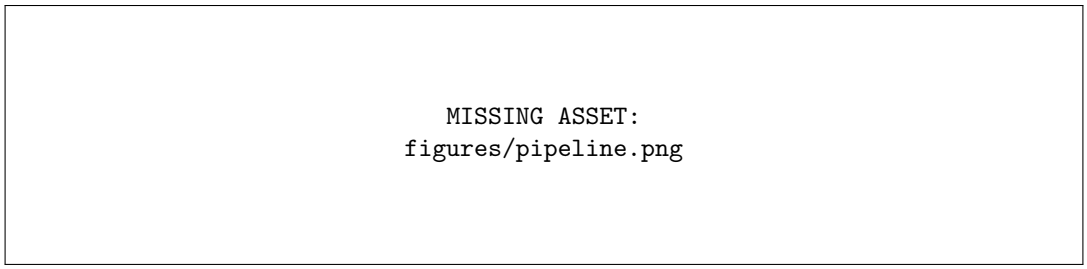


Figure 1: Processing pipeline from upload to persisted results.

deterministic synthetic detector is available for development and tests; it is not used for the real-video results reported here.

3.3 Tracking and Camera-Motion Compensation

Three tracker back-ends share one interface and are selected per job: a *mock* tracker (deterministic centre-distance association, used as a simple baseline), ByteTrack [22], and TAPNext [23], which follows point queries through the video. The tracker match threshold τ_{match} (default 120 px) and the number of buffer frames before an unmatched track is considered lost (default 30) are configurable.

When compensation is enabled, boxes are translated into a stabilised coordinate system before the tracker update and mapped back to image coordinates afterwards, so camera motion is not seen as object motion. **[Author check:** the source describes the estimator inconsistently: the pipeline section describes a synthetic per-mode model (`static`/`drift`), whereas the evaluation describes a measured shift taken as the median displacement of detection centres. The ablation in Section 5.3 is described by the source as using the measured shift.**]**

3.4 Identity Memory and Re-Identification

For each tracked detection the pipeline computes an appearance embedding. Each identity k keeps a unit-norm prototype $\mathbf{p}_k$, updated as a moving average of the embeddings $\mathbf{e}_t$ assigned to it with memory update rate α (default 0.2):

$$\mathbf{p}_k \leftarrow \frac{(1-\alpha)\mathbf{p}_k + \alpha \mathbf{e}_t}{\|(1-\alpha)\mathbf{p}_k + \alpha \mathbf{e}_t\|_2}. \tag{3}$$

The quality of an observation is the detector confidence, multiplied by 0.55 if the detection is flagged as occluded; observations with quality below 0.3 are discarded before they can influence a prototype. An identity's quality is an exponential moving average of its accepted observations' qualities, and the identity is marked confident once it has at least three observations and quality of at least 0.5.

Identity numbers are assigned per track chain. A new track receives the lowest unused identity number unless it is a plausible reappearance of a track that was reported lost, in which case it inherits that identity. Candidates are ranked by cosine similarity, which must exceed the ReID threshold τ_{reid} (default 0.6), and are then filtered by a motion gate: the reappearing track must lie inside a radius that grows with the length of the occlusion (default parameters reported as 120 px and 45 px per frame). Accepted and rejected links are stored as re-identification events with their similarity. Tracks that overlap in time can never share an identity, which prevents two simultaneously visible fruit from being merged. The converse failure remains possible: a missed reappearance creates a new identity and therefore *over*-counts, while a false link across a long gap merges two fruit and under-counts. The threshold τ_{reid} trades these two errors against each other.

Embedder. In real mode the embedding is either a checkpoint at a configured path or, when none is present, a torchvision ResNet-18 feature extractor. The source report is inconsistent about whether embeddings were trained: one section describes contrastive (InfoNCE) training [16] with pseudo-labels from density-based clustering [7], whereas the evaluation describes an untrained ResNet-18 embedder and the conclusion lists fine-tuning as future work. We therefore do not claim that a trained embedder produced any result in this paper. **[Author check:** state which embedder was used in each experiment, and whether the InfoNCE training described in the source was actually run. Also verify the citation `reddi2017` as the source for the InfoNCE loss; I could not confirm it from the supplied material.**]** Identity prototypes can be inspected in two dimensions with t-SNE [19] or, for few identities, a principal-component projection [9].

3.5 Evaluation Tooling

The evaluation module implements IDF1, HOTA, identity switches, track fragmentation, count error, and identity recovery rate. Two evaluation paths exist. The API experiment endpoint runs a synthetic orchard scenario with the mock detector, and therefore exercises the tracking and metric code rather than real fruit detection. Real-video results in this paper come from offline scripts that run the YOLO detector and a tracker and score detection and tracking only; *the identity memory is not part of the evaluated loop.*

4 Experimental Setup

4.1 Data

Table 1 lists the three orchard videos used.

Table 1: Evaluation videos. The source report marks this table "to be filled in" although it contains values; treat it as provisional.

Video	Source file	Camera*	Resolution	FPS	Frames	GT identities†
V1	`test`	static	960×506	29.9	747	9
V2	`test4`	static	1366×768	24	141	7
V3	`Test3`	static	768×432	24	337	14

*The "static" label is as printed in the source, but the source also reports non-trivial global camera motion on these videos (Section 5.4). †Machine-generated; see Section 4.3.

[Author check: the earlier manuscript listed 39 GT fruit for V1; the GT construction table and the counting table both give 9, so 9 is used here. Confirm against the original Table 8.1, and confirm the camera label.**]**

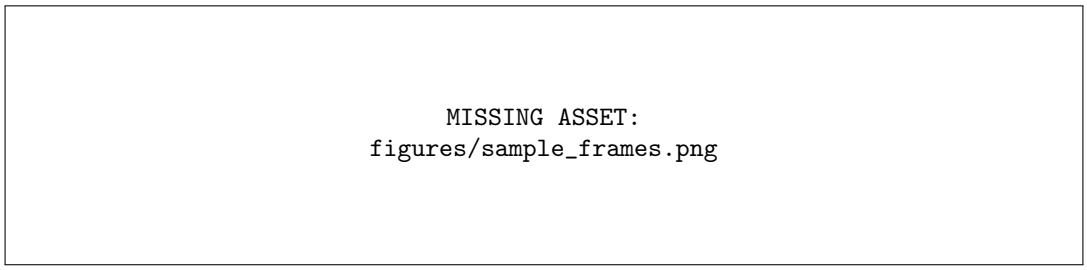


Figure 2: Sample frames from V1–V3.

4.2 Hardware

Tracking and runtime measurements were taken on one machine: Apple M2 Max (12-core CPU, 30-core GPU), 64 GB unified memory, macOS (Darwin 24, arm64), Python 3.12, PyTorch 2.14, Ultralytics 8.4. The detector training runs of Table 3 were executed on a separate Windows workstation whose specification was not logged. **[Author check:** the source says both "12 CPU cores" and "12 performance cores"; the latter should be checked against the machine specification.**]**

4.3 Provisional Ground Truth

The ground truth is **machine-generated and has not been confirmed by a human annotator**. It was built by running the same YOLO detector as the system over every frame, associating boxes at the full frame rate, and keeping identities that last at least fifty frames. The minimum-length rule removes the short tracks that occlusion and false positives create: on V1, raw association yielded 61 identities, of which 44 lasted fewer than twenty frames and nine lasted fifty frames or more. Table 2 summarises the result.

Table 2: Construction of the provisional ground truth: raw association output and identities kept by the minimum-length rule.

Video	Raw ids	Kept	Dropped	Boxes	Max concurrent
V1	61	9	52	2956	10
V2	44	7	37	854	10
V3	43	14	29	1947	12

Two consequences follow. First, because the reference comes from the same detector the system runs, the identity metrics measure how consistently a tracker maintains identities that the detector proposes; they say nothing about whether the detector is right, and they are *not* absolute counting accuracy. Second, the reference is unreviewed: frames were exported every tenth frame for manual review, but that review has not been reported. **[Author check:** Table 2 lists a maximum concurrency of 10 for V1 and V2, which exceeds the 9 and 7 kept identities. Either concurrency was measured on unfiltered tracks or the retained identities under-count the visible fruit. The source's claim that the numbers are consistent with the concurrency bound is therefore not repeated here. Also state whether the detector's training or validation images overlap V1–V3.**]**

4.4 Metrics

For each tracker we report IDF1, HOTA, and the number of identity switches, averaged over V1–V3. For counting, we compare the number of predicted tracks with the number of provisional ground-truth identities. **[Author check:** the source labels the count-error column of the tracker table as a percentage but defines it as a mean absolute difference between track and identity counts; its unit is unresolved and is left unlabelled below.**]**

5 Results

5.1 Detector Training Runs

Four detector training runs were logged by Ultralytics (Table 3): YOLOv8n (50 epochs), two YOLOv8m runs (10 and 93 epochs), and YOLOv8x (40 epochs). The YOLOv8n run reached the

highest validation $mAP_{50\text{-}95}$ (0.827) and the lowest latency (33.6 ms). It is the default detector of the deployment. The runs are not a controlled comparison: the YOLOv8n and YOLOv8m runs were trained on a second, larger annotation set with different schedules, and the YOLOv8x run was logged only at its final epoch. The table therefore justifies the deployment default but does not rank the architectures. The differences in $mAP_{50\text{-}95}$ (0.788–0.827) are small and are confounded with dataset and schedule.

Table 3: Detector training runs. P, R, and mAP are validation metrics at the best epoch. Latency is the median of ten forward passes of the exported weights at 640 pixels.

Run	Best ep.	P	R	mAP50	mAP50-95	ms/frame
mango_yolov8n-4	49	0.946	0.975	0.985	0.827	33.6
mango_yolov8m_exp-2	10	0.928	0.949	0.979	0.808	83.5
mango_yolov8m_exp-3	73	0.958	0.971	0.987	0.789	83.5
mango_yolov8x	40	0.958	0.961	0.985	0.788	173.1

[Author check: the source prose cites exported weight sizes, but no weight-size column exists in the table, so sizes are omitted here. The device and hardware for the latency column are unclear (the source ties it to the evaluation machine), and 33.6 ms for the detector alone exceeds the 25.3 ms per frame reported for detection plus tracking in Table 7. Reconcile the two.**]**

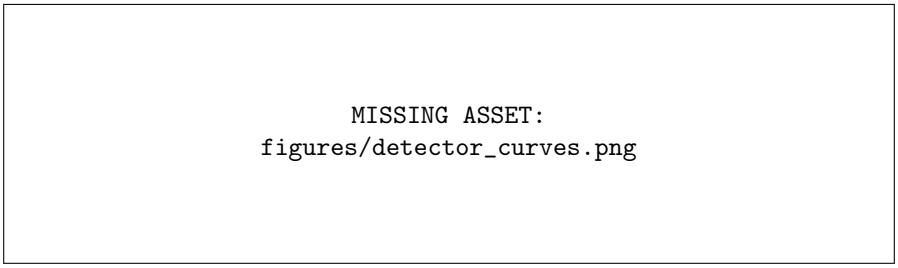


Figure 3: Validation $mAP_{50\text{-}95}$ per epoch for the two longest detector runs.

5.2 Tracker Comparison

Table 4 compares the three tracker back-ends against the provisional ground truth.

Table 4: Tracker comparison, averaged over V1–V3, against the provisional full-rate ground truth. Count error is the source's mean absolute difference between predicted tracks and ground-truth identities; its unit is unresolved.

Tracker	IDF1	HOTA	ID switches	Count error
mock (baseline)	70.9	75.9	1.0	0.67
ByteTrack	69.0	73.0	5.7	0.61
TAPNext‡	0.88	0.89	3.2	0.55

‡Unverified. The source column headings say "(%)", but the TAPNext IDF1 and HOTA are printed as 0.88 and 0.89 while the other rows are on a 0–100 scale. Whether TAPNext is on a 0–1 scale, is a percentage, or is a different quantity is unconfirmed, so the TAPNext row is neither converted nor compared with the other rows.

[Author check: confirm the scale of IDF1 and HOTA for every row. A previous draft transcribed the TAPNext row as 0.8, 80.8, and 93.2; the source values 0.88, 0.89, 3.2, and 0.55 are

restored here. Table 6 implies a mean relative count error of 0.63 for ByteTrack ($= (0.111 + 1.571 + 0.214)/3$), not 0.61, so the two tables may come from different runs. The ByteTrack IDF1 here (69.0) also differs from the camera-motion "off" row (68.9).]

MISSING ASSET:
figures/tracker_comparison.png

Figure 4: Per-video IDF1, HOTA, and identity switches by tracker. **[Author check:** the source caption refers to two evaluated trackers while the table has three; confirm which are plotted.]

5.3 Camera-Motion Compensation

Table 5 reports the aggregate effect of compensation. With compensation, mean IDF1 changed from 68.9 to 69.2 and mean HOTA from 73.0 to 73.6, while mean identity switches rose from 5.7 to 17.3. The identity-quality metrics are therefore essentially unchanged and the switch count is higher. We do not draw a causal conclusion about why: the source's per-video statement on this point (that switches "lower from 9 to 36" on V1) is self-contradictory and per-video values are not reported here.

Table 5: Effect of compensating camera motion, averaged over V1–V3. Per the source, the shift of each frame is the median displacement of its detection centres; boxes are moved into stabilised coordinates before tracking and back before storage.

Camera-motion mode	IDF1 (%)	HOTA (%)	ID switches
Off	68.9	73.0	5.7
On (compensation applied)	69.2	73.6	17.3

[Author check: resolve the per-video statement "from 9 to 36" for V1; the aggregate values above are preserved unchanged.]

MISSING ASSET:
figures/camera_motion_ablation.png

Figure 5: Identity continuity with and without camera-motion compensation.

5.4 Counting Accuracy

Table 6 compares the ByteTrack track count with the provisional ground-truth identity count. ByteTrack over-counted on every video, by 1, 11, and 3 tracks.

Table 6: Count accuracy for ByteTrack against the provisional ground truth (retained identities, Table 2).

Video	GT identities	Predicted tracks	Absolute error	Error (%)
V1	9	10	1	11.1
V2	7	18	11	157.1
V3	14	17	3	21.4

V2 is the hard case: ByteTrack produced more than twice as many tracks as there are ground-truth identities. The source reports that the median track lasts 377 frames on V1 but 37 frames on V2, a clip of only 141 frames, and that V2 has a larger global camera shift (median 4.5 px against 0.3 px on V1; 95th percentile 44.7 px, as reported). Those figures are consistent with fragmented tracks on V2, but our evidence does not separate camera motion from other causes. Two caveats apply to all three rows: the reference is machine-generated (Section 4.3), and the source does not state whether the predicted track count applies a minimum-length filter comparable to the one used to build the reference. If it does not, part of the excess may reflect the filter rather than tracker fragmentation.

5.5 Sensitivity to the Detection Threshold

The detection-confidence threshold is the one threshold the offline harness can sweep end to end. Each video was re-run with ByteTrack at three thresholds between 0.25 and 0.75. The source reports that the identity metrics are not measurably affected, while the predicted track count falls sharply as low-confidence detections are removed; it attributes the insensitivity of the metrics to ByteTrack's own high-confidence threshold. We report this as an observation from the source and have not independently verified the mechanism. **[Author check:** the source text describes ByteTrack only, whereas the figure caption says both trackers were swept.**]**

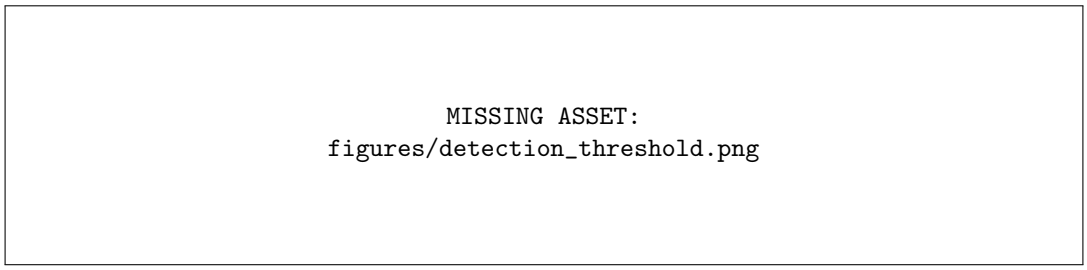


Figure 6: Predicted track count and count error against detection confidence threshold on V1–V3.

5.6 Runtime

Table 7 summarises per-frame processing time over V1–V3. The time covers detection and tracking and excludes frame decoding; embedding extraction and identity matching are not included.

Table 7: Processing time per frame, averaged over V1–V3, on the machine described in Section 4.

Tracker	Mean (ms/frame)	95th pct (ms)	Throughput (FPS)
mock	25.8	28.8	39.0
ByteTrack	25.3	28.6	39.5
TAPNext‡	600.0	700.0	1.67

‡Unverified; the round values (600.0, 700.0) should be confirmed as measurements. The TAPNext row is not interpreted.

For the mock and ByteTrack back-ends, detection plus tracking runs at about 39 frames per second, above the 24–29.9 FPS of the evaluation videos, on this machine. The cost of the embedder and identity memory is not measured.

MISSING ASSET:
figures/per_frame_time.png

Figure 7: Per-frame detection and tracking time over the three evaluation videos.

5.7 Unresolved: ReID Threshold Sensitivity and Identity Projection

The source states that the ReID-threshold sweep requires the identity memory to be in the evaluated loop, that the offline harness currently scores detection and tracking only, and that the sweep therefore cannot yet be run. The source nevertheless contains a populated table, whose caption does not match its column headings. Its provenance is unconfirmed, so Table 8 reproduces the printed values for transparency only; **they are not evidence** and nothing in this paper depends on them. Likewise, we make no claim about how well identity prototypes separate (Figure 8): that depends on which embedder was used (Section 3), which is unresolved. The source's illustrative re-identification example refers to a video labelled V4, which is not in the evaluation set (Table 1), and is omitted.

Table 8: **Unresolved; provenance unconfirmed.** ReID-threshold values as printed in the source report (averaged over V1–V3). The source caption describes different columns from these headings. Do not cite as a result.

ReID threshold	Identities found	IDF1	ID switches
0.90	13.3	3.7	11
0.93	14.3	2.7	12
0.95	15.7	1.3	13
0.97	17.0	0.0	17

[Author check: a previous draft printed ID switches as 110, 120, 130, 170; the source values are 11, 12, 13, 17. Decide whether to run the experiment or delete the table.**]**

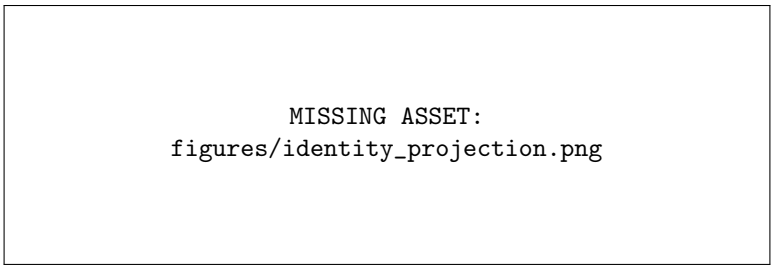


Figure 8: Two-dimensional projection of the identity prototypes of V2 (unresolved; not interpreted).

5.8 Discussion

We restrict interpretation to results that the provisional ground truth and the transcribed tables can support.

Pipeline feasibility. Detection plus ByteTrack tracking runs faster than the frame rate of the evaluation videos on a laptop-class machine, so the pipeline is not computationally prohibitive at the detection-and-tracking level. This says nothing about the cost of the embedder or identity memory.

Baseline versus ByteTrack. On these three videos the centre-distance baseline scored slightly higher (IDF1 70.9 against 69.0, HOTA 75.9 against 73.0) with fewer identity switches (1.0 against 5.7). The gaps are a few points, from three videos with no variance estimate and a reference built by the same detector. We therefore do not conclude that either is better; the data do not show a benefit of ByteTrack over a simple association baseline on this reference.

Counting. All three counts were above the reference, and by a large margin on V2 (Table 6). Over-counting is the direction expected from identity fragmentation. Since the reference removes short tracks and may itself under-count (Section 4.3), the data do not tell us how much of the error is tracker fragmentation, reference construction, or camera motion.

Camera-motion compensation. Compensation gave no clear benefit: IDF1 and HOTA moved by 0.3 and 0.6 points while identity switches rose from 5.7 to 17.3 on average. The conflicting per-video statement in the source prevents a finer reading.

Detection threshold. The reported insensitivity of the identity metrics to the confidence threshold (with a sharp change in track count) suggests the threshold mainly affects how many tracks are created, but the scope of this observation (ByteTrack only, or both trackers) is unresolved.

What cannot be concluded. Because the identity memory is not in the evaluated loop, the results say nothing about whether re-identification reduces double-counting; because the ReID-threshold sweep and the TAPNext results are unverified, they support no conclusion; and because evaluation covers three short videos from a single recording condition, they do not support claims about generalisation.

6 Limitations

- **Provisional ground truth.** The reference is machine-generated from the same detector and not human-verified; identity metrics measure consistency with it, not absolute counting accuracy. Reference counts may under-count (Table 2).
- **Identity memory not evaluated.** The offline harness scores detection and tracking only. The system's central mechanism, prototype-based re-identification, has no validated result here.
- **Embedder unresolved.** Whether any embedder was trained is unconfirmed (Section 3).
- **Unresolved results.** The ReID-threshold values and the TAPNext metrics (scale and units) are unverified and are not interpreted.
- **Small, internally inconsistent dataset.** Three short videos, no variance estimates; Table 1 is flagged incomplete in the source, and the camera label conflicts with the reported camera motion.
- **Scope of identity matching.** Only short-term re-identification within a video is implemented. Matching fruit across sessions or days is not implemented or evaluated.
- **Deployment.** The system uses SQLite and local-disk storage; it has not been evaluated under concurrent multi-user load.

7 Conclusion and Future Work

We presented OrchardVision, an implemented pipeline for fruit tracking in orchard video that combines detection, interchangeable trackers, optional camera-motion compensation, and a prototype-based identity memory, with all intermediate results persisted and a harness for tracker comparison. On three videos and a provisional, machine-generated reference, ByteTrack over-counted on all videos, a simple baseline performed comparably to it, and camera-motion compensation gave no clear benefit. These findings are about tracker consistency; absolute counting accuracy, the benefit of the identity memory, and the TAPNext and ReID-threshold results remain unvalidated.

Future work, in order of priority: (i) human verification of the ground truth and a larger, annotated, multi-season dataset; (ii) running the identity memory inside the evaluated loop, including the ReID-threshold sweep and a clear statement of which embedder is used; (iii) fine-tuning the embedder on fruit crops with a metric-learning loss; (iv) clarifying and re-running the TAPNext evaluation; (v) linking identities across recording sessions to follow fruit through a season, which is not implemented here; and (vi) depth or multi-camera information to resolve mutually occluding fruit.

References

[1] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. Bot-sort: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*, 2022.

[2] S. Bargoti and J. Underwood. Deep fruit detection in orchards. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3626–3633. IEEE, 2017.

[3] M. Bayer. Sqlalchemy. In A. Brown and G. Wilson, editors, *The Architecture of Open Source Applications, Volume II*. aosabook.org, 2012.

[4] K. Bernardin and R. Stiefelhagen. Evaluating multiple object tracking performance: The clear mot metrics. *EURASIP Journal on Image and Video Processing*, 2008:Article 246309, 2008.

[5] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft. Simple online and realtime tracking. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468. IEEE, 2016.

[6] C. Doersch, A. Gupta, L. Markeeva, A. Recasens, L. Smaira, Y. Aytar, J. Carreira, A. Zisserman, and Y. Yang. Tap-vid: A benchmark for tracking any point in a video. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[7] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231. ACM, 1996.

[8] A. Hermans, L. Beyer, and B. Leibe. In defense of the triplet loss for person re-identification. *arXiv preprint arXiv:1703.07737*, 2017.

[9] I. T. Jolliffe and J. Cadima. Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2015.02052), 2016.

[10] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[11] A. Koirala, K. B. Walsh, Z. Wang, and C. McCarthy. Deep learning — method overview and review of use for fruit detection and yield estimation. *Computers and Electronics in Agriculture*, 162:219–234, 2019.

[12] H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

[13] X. Liu, S. W. Chen, C. Liu, S. S. Shivakumar, J. Das, C. J. Taylor, J. Underwood, and V. Kumar. Monocular camera based fruit counting and mapping with semantic data association. *IEEE Robotics and Automation Letters*, 4(3):2296–2303, 2019.

[14] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixe, and B. Leibe. Hota: A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2):548–578, 2021.

[15] S. Ramirez. Fastapi [computer software]. https://fastapi.tiangolo.com. No date given in the source report.

[16] R. Reddi, G. Kreibich, N. Poffenbarger, and R. Chandra. A loss function for deep metric learning. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 4187–4195. IEEE, 2017.

[17] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788. IEEE, 2016.

[18] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi. Performance measures and a data set for multi-target, multi-camera tracking. In *European Conference on Computer Vision Workshops*, pages 17–35. Springer, 2016.

[19] L. van der Maaten and G. Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(Nov):2579–2605, 2008.

[20] N. Wojke, A. Bewley, and D. Paulus. Simple online and realtime tracking with a deep association metric. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, pages 3645–3649. IEEE, 2017.

[21] YOLO Team and Ultralytics. Ultralytics yolo [computer software]. https://github.com/ultralytics/ultralytics, 2024.

[22] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. Bytetrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, pages 1–21. Springer, 2022.

[23] A. Zholus, C. Doersch, Y. Yang, S. Koppula, V. Patraucean, X. O. He, I. Rocco, M. S. M. Sajjadi, S. Chandar, and R. Goroshin. Tapnext: Tracking any point (tap) as next token prediction. *arXiv preprint arXiv:2504.05579*, 2025.